Conteúdo completo do diretório: DrTilapIA
==========
=======

Diretório: C:\Users\usuario\Python\DrTilapIA

Arquivo: Estrutura_dir.py
Conteúdo:

```
from pathlib import Path

# Diretórios considerados "lixo" em projetos reais
IGNORAR_DIRS = {
    ".git",
    ".idea",
    ".vscode",
    "__pycache__",
    "node_modules",
    ".pytest_cache",
    "dist",
    "build",
    "venv",
    "env",
}

# Arquivos inúteis também podem ser ignorados aqui
IGNORAR_ARQUIVOS = {
    ".DS_Store",
    "Thumbs.db",
}


def gerar_arvore(diretorio, prefixo=""):
    caminho = Path(diretorio)

    # Lista somente itens relevantes
    itens = []
    for item in caminho.iterdir():

        # Ignorar ocultos (.qualquercoisa)
        if item.name.startswith(".") and item.name != ".gitignore":
            continue

        # Ignorar diretórios lixo
        if item.is_dir() and item.name in IGNORAR_DIRS:
            continue

        # Ignorar arquivos lixo
        if item.is_file() and item.name in IGNORAR_ARQUIVOS:
            continue

        itens.append(item)

    # Ordenação: pastas primeiro, depois arquivos
    itens.sort(key=lambda x: (not x.is_dir(), x.name.lower()))

    total = len(itens)
    for index, item in enumerate(itens):
```

```
        ultimo = (index == total - 1)
        conector = "└── " if ultimo else "├── "

        print(prefixo + conector + item.name)

        if item.is_dir():
            novo_prefixo = prefixo + ("    " if ultimo else "│   ")
            gerar_arvore(item, novo_prefixo)


# --- Execução ---
print("📦 Estrutura do Projeto (clean):")
gerar_arvore(".")
==================================================

Arquivo: Retorna_conteudo.py
Conteúdo:
import os


def gerar_relatorio(diretorio_raiz, arquivo_saida, extensoes=None):
    """
    extensoes: tupla/lista de extensões, ex: ('.py', '.js', '.ts')
    se extensoes for None, pega todos os arquivos
    """
    if extensoes is not None:
        extensoes = tuple(extensoes)

    with open(arquivo_saida, 'w', encoding='utf-8') as f:
        f.write(f"Conteúdo completo do diretório:
{os.path.basename(diretorio_raiz)}\n")
        f.write("=" * 80 + "\n\n")

        for root, dirs, files in os.walk(diretorio_raiz):
            # Ignora diretórios que começam com "."
            dirs[:] = [d for d in dirs if not d.startswith('.')]

            rel_path = os.path.relpath(root, diretorio_raiz)
            f.write(f"Diretório: {rel_path if rel_path != '.' else
diretorio_raiz}\n")
            f.write("-" * 50 + "\n")

            for file in sorted(files):
                if extensoes is not None and not
file.endswith(extensoes):
                    continue

                caminho_completo = os.path.join(root, file)
                caminho_rel = os.path.relpath(caminho_completo,
diretorio_raiz)
                f.write(f"\nArquivo: {caminho_rel}\n")
                f.write("Conteúdo:\n")
                try:
                    with open(caminho_completo, 'r', encoding='utf-8') as
arquivo:
                        conteudo = arquivo.read()
                        f.write(conteudo)
                except UnicodeDecodeError:
```

```
                    f.write("[Erro: arquivo binário ou encoding inválido
- conteúdo omitido]\n")
                except Exception as e:
                    f.write(f"[Erro ao ler: {str(e)}]\n")
                f.write("\n" + "=" * 50 + "\n")


# Uso
diretorio = r'C:\Users\usuario\Python\DrTilapIA'
extensoes_desejadas = ('.py', '.yml', '.Dockerfile', )
nome_saida = f"{os.path.basename(diretorio)}_relatorio_completo.txt"

gerar_relatorio(diretorio, nome_saida, extensoes=extensoes_desejadas)
print(f"Relatório gerado: {nome_saida}")
==================================================

Arquivo: docker-compose.yml
Conteúdo:
services:
  # Serviço do Backend (Cérebro do Dr. Tilápia)
  api:
    build:
      context: .
      dockerfile: backend/Dockerfile
    container_name: dr_tilapia_api
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
      - ./.env:/app/.env
    env_file:
      - .env
    restart: always
    networks:
      - tilapia_net

  # Serviço do Frontend (Rosto do Dr. Tilápia)
  frontend:
    build:
      context: .
      dockerfile: frontend.Dockerfile
    container_name: dr_tilapia_ui
    ports:
      - "3000:3000"
      - "8001:8001"
    volumes:
      - ./frontend:/app
    environment:
      - API_URL=http://localhost:8000
    depends_on:
      - api
    restart: always
    networks:
      - tilapia_net

networks:
  tilapia_net:
    driver: bridge
```

==========

Arquivo: frontend.Dockerfile
Conteúdo:

```
# Usamos Python 3.11 para estabilidade total com Reflex
FROM python:3.11-slim

WORKDIR /app

# Instala dependências de sistema essenciais
RUN apt-get update && apt-get install -y \
    curl \
    unzip \
    && rm -rf /var/lib/apt/lists/*

# 1. Copia o arquivo de requisitos
COPY frontend/requirements.txt .

# 2. Instala as dependências bases
RUN pip install --no-cache-dir \
    sqlalchemy==2.0.29 \
    sqlmodel==0.0.16

# 3. Aguarda um segundo para garantir que o pip finalizou
RUN sleep 1

# 4. Agora instala o resto com o ambiente blindado
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copia o código do frontend
COPY frontend/ .

# 6. NÃO rodamos reflex init aqui. Ele será executado no container em
runtime.
# Isso evita o conflito de metaclasses que ocorre durante o build.

# Portas: 3000 (UI) e 8001 (Eventos do Reflex)
EXPOSE 3000 8001

# Comando de execução: O reflex run já cuida de chamar reflex init se
necessário
CMD ["reflex", "run", "--env", "prod", "--backend-port", "8001"]
```

==========
Diretório: backend

Arquivo: backend\backend.Dockerfile
Conteúdo:

```
FROM python:3.11-slim

WORKDIR /app

# Instala dependências de sistema
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*
```

```
# 1. Copia o arquivo de requisitos
COPY backend/requirements.txt .

# 2. Instala as versões "bloqueadas" primeiro
RUN pip install --no-cache-dir \
    sqlalchemy==2.0.29 \
    sqlmodel==0.0.16

# 3. Aguarda para garantir consistência
RUN sleep 1

# 4. Instala o resto
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copia o código
COPY backend/ .

EXPOSE 8000

# Comando de execução do motor de IA
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "-
-reload"]
==================================================

Arquivo: backend\create_standard_user.py
Conteúdo:
import bcrypt
from app.database import supabase


def create_user(email, password, name, role):
    # Gerando Hash seguro com Bcrypt
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-
8')

    data = {
        "email": email,
        "hashed_password": hashed,
        "full_name": name,
        "role": role  # 'admin' ou 'user'
    }

    try:
        # Insere na tabela public.users
        res = supabase.table("users").insert(data).execute()
        print(f"✅ Usuário {email} criado com sucesso como '{role}'!")
    except Exception as e:
        print(f"❌ Erro ao criar usuário: {e}")


if __name__ == "__main__":
    # Criando o usuário padrão solicitado
    create_user("usuario@gmail.com", "123456", "Usuário Comum", "user")
==================================================

Arquivo: backend\criar_admin.py
Conteúdo:
```

```
import os
import sys

# 1. Localiza a pasta 'backend' de forma absoluta
# Isso evita problemas com o OneDrive e caminhos relativos no Windows
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# 2. Injeta o caminho no sys.path para o Python "enxergar" a pasta app
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)

try:
    # Importação direta (agora o Python sabe que 'app' está na raiz do
sys.path)
    from app.database import supabase
    from app.auth.auth_service import get_password_hash


    def main():
        print("\n" + "=" * 50)
        print("   DR. TILÁPIA - CONFIGURAÇÃO DE ACESSO ADMIN")
        print("=" * 50)

        email = input("📧 E-mail do Administrador: ").strip()
        senha = input("🔑 Senha do Administrador: ").strip()

        if not email or not senha:
            print("❌ Erro: Campos obrigatórios vazios.")
            return

        print("\n⏳ Gerando Hash e conectando ao Supabase...")

        try:
            # Gerando o hash usando sua função do auth_service
            senha_hash = get_password_hash(senha)

            # Inserindo na tabela 'users'
            # IMPORTANTE: Verifique se as colunas no Supabase são
exatamente estas
            dados = {
                "email": email,
                "hashed_password": senha_hash,
                "role": "admin"
            }

            response = supabase.table("users").insert(dados).execute()

            if response.data:
                print(f"\n✅ SUCESSO! Usuário '{email}' criado.")
                print("🚀 Agora você pode logar no Frontend (Porta
3000).")
            else:
                print("\n❌ Erro: O banco não retornou confirmação.")

        except Exception as e:
            print(f"\n❌ Erro na operação: {e}")
```

```
    if __name__ == "__main__":
        main()

except ImportError as e:
    print(f"\n❌ ERRO DE IMPORTAÇÃO: {e}")
    print(f"DEBUG: Verifique se existe o arquivo: {os.path.join(BASE_DIR,
'app', 'database.py')}")
    print(f"DEBUG: Verifique se existe o arquivo: {os.path.join(BASE_DIR,
'app', '__init__.py')}")
==================================================

Arquivo: backend\criar_usuario_admin.py
Conteúdo:
import os
import logging
import re
from supabase import create_client, Client
from app.services.user_service import UserService
from dotenv import load_dotenv
from typing import List, Dict, Any, Optional

# Carrega variáveis de ambiente do arquivo .env
load_dotenv()

# Configuração de logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s -
%(levelname)s - %(message)s')
logger = logging.getLogger(__name__)


class AdminCLI:
    """
    Interface de linha de comando para gerenciar usuários do Supabase
Auth.
    Utiliza o UserService para operações administrativas.
    """

    def __init__(self):
        try:
            self.user_service = UserService()
            logger.info("AdminCLI inicializado com UserService.")
        except ValueError as e:
            logger.critical(f"Erro de inicialização do UserService: {e}")
            print(f"\n❌ ERRO CRÍTICO: {e}")
            print(
                "   Certifique-se de que SUPABASE_URL e
SUPABASE_SERVICE_ROLE_KEY estão definidos no seu arquivo .env.")
            exit(1)

    def _validate_email(self, email: str) -> bool:
        """Valida o formato do email."""
        if not re.fullmatch(r"[^@]+@[^@]+\.[^@]+", email):
            print("❌ Formato de email inválido.")
            return False
        return True

    def _validate_password_strength(self, password: str) -> bool:
```

```
        """Valida a força da senha (mínimo 8 caracteres, maiúscula,
minúscula, número, caractere especial)."""
        if len(password) < 8:
            print("❌ A senha deve ter no mínimo 8 caracteres.")
            return False
        if not re.search(r"[A-Z]", password):
            print("❌ A senha deve conter pelo menos uma letra
maiúscula.")
            return False
        if not re.search(r"[a-z]", password):
            print("❌ A senha deve conter pelo menos uma letra
minúscula.")
            return False
        if not re.search(r"\d", password):
            print("❌ A senha deve conter pelo menos um número.")
            return False
        if not re.search(r"[!@#$%^&*(),.?\"{}\|<>]", password):
            print("❌ A senha deve conter pelo menos um caractere
especial.")
            return False
        return True

    def _get_user_input(self, prompt: str, sensitive: bool = False) ->
str:
        """Obtém entrada do usuário, com opção para entrada sensível
(senha)."""
        if sensitive:
            return input(prompt)
        return input(prompt)

    def display_menu(self):
        """Exibe o menu de opções para o usuário."""
        print("\n--- Gerenciamento de Usuários Supabase ---")
        print("1. Criar Novo Usuário")
        print("2. Listar Usuários Existentes")
        print("3. Deletar Usuário")
        print("4. Sair")
        print("")

    def handle_create_user(self):
        """Processa a criação de um novo usuário."""
        print("\n--- Criar Novo Usuário ---")
        while True:
            email = self._get_user_input("Email do novo usuário:
").strip()
            if self._validate_email(email):
                break

        existing_user = self.user_service.get_user_by_email(email)
        if existing_user:
            print(f"⚠️ Usuário com email '{email}' já existe. ID:
{existing_user.get('id')}")
            return

        while True:
            password = self._get_user_input("Senha do novo usuário: ",
sensitive=True).strip()
```

```
            if self._validate_password_strength(password):
                break

        name = self._get_user_input("Nome completo do usuário: ").strip()

        roles = ["admin", "user", "guest"]
        while True:
            role = self._get_user_input(f"Role do usuário ({', 
'.join(roles)}): ").strip().lower()
            if role in roles:
                break
            print(f"❌ Role inválida. Escolha entre: {', '.join(roles)}")

        try:
            new_user = self.user_service.create_user(email, password, 
name, role=role)
            print(f"\n🎉 Usuário '{email}' criado com sucesso!")
            print(f"   ID: {new_user.get('id')}")
            print(f"   Nome: {new_user.get('user_metadata', 
{}).get('name')}")
            print(f"   Email: {new_user.get('email')}")
            print(f"   Role: {new_user.get('user_metadata', 
{}).get('role')}")
        except Exception as e:
            logger.error(f"Erro ao criar usuário '{email}': {e}", 
exc_info=True)
            print(f"\n❌ ERRO ao criar usuário: {e}")

    def handle_list_users(self):
        """Processa a listagem de usuários existentes."""
        print("\n--- Listar Usuários ---")
        try:
            users = self.user_service.list_users()
            if not users:
                print("Nenhum usuário encontrado.")
                return

            print(f"Total de usuários: {len(users)}")
            for user in users:
                user_id = user.get('id', 'N/A')
                email = user.get('email', 'N/A')
                name = user.get('user_metadata', {}).get('name', 'N/A')
                role = user.get('user_metadata', {}).get('role', 'N/A')
                created_at = user.get('created_at', 'N/A')
                print(f"")
                print(f"ID: {user_id}")
                print(f"Email: {email}")
                print(f"Nome: {name}")
                print(f"Role: {role}")
                print(f"Criado em: {created_at}")
            print(f"")
        except Exception as e:
            logger.error(f"Erro ao listar usuários: {e}", exc_info=True)
            print(f"\n❌ ERRO ao listar usuários: {e}")

    def handle_delete_user(self):
        """Processa a exclusão de um usuário."""
```

```
        print("\n--- Deletar Usuário ---")
        user_id = self._get_user_input("ID do usuário a ser deletado:
").strip()
        if not user_id:
            print("❌ ID do usuário não pode ser vazio.")
            return

        try:
            user_to_delete = self.user_service.get_user_by_id(user_id)
            if not user_to_delete:
                print(f"⚠️ Usuário com ID '{user_id}' não encontrado.")
                return

            confirm = self._get_user_input(
                f"Tem certeza que deseja deletar o usuário
'{user_to_delete.get('email')}' (s/N)? ").strip().lower()
            if confirm == 's':
                self.user_service.delete_user(user_id)
                print(f"\n✅ Usuário '{user_id}' deletado com sucesso.")
            else:
                print("Operação de exclusão cancelada.")
        except Exception as e:
            logger.error(f"Erro ao deletar usuário '{user_id}': {e}",
exc_info=True)
            print(f"\n❌ ERRO ao deletar usuário: {e}")

    def run(self):
        """Executa o loop principal da interface de linha de comando."""
        while True:
            self.display_menu()
            choice = self._get_user_input("Escolha uma opção: ").strip()

            if choice == '1':
                self.handle_create_user()
            elif choice == '2':
                self.handle_list_users()
            elif choice == '3':
                self.handle_delete_user()
            elif choice == '4':
                print("Saindo do gerenciador de usuários. Até mais!")
                break
            else:
                print("Opção inválida. Por favor, tente novamente.")


if __name__ == "__main__":
    cli = AdminCLI()
    cli.run()
==================================================

Arquivo: backend\docker-compose.yml
Conteúdo:
services:
  api:
    build:
      context: ..
      dockerfile: backend/Dockerfile
```

```
    container_name: dr_tilapia_api_dev
    ports:
      - "8000:8000"
    volumes:
      - .:/app
      - ../.env:/app/.env
    env_file:
      - ../.env
    restart: always

# Nota: Este compose é focado em desenvolvimento isolado do motor de IA.
==================================================
Diretório: backend\app
--------------------------------------------------

Arquivo: backend\app\__init__.py
Conteúdo:
"""
Pacote principal da aplicação DrTilápia 1.3.
Centraliza importações de módulos críticos.
"""

__version__ = "2.0"
==================================================

Arquivo: backend\app\database.py
Conteúdo:
import os
from supabase import create_client, Client
from dotenv import load_dotenv
from pathlib import Path

# Carrega o .env da raiz do backend
env_path = Path(__file__).resolve().parent.parent.parent / '.env'
load_dotenv(dotenv_path=env_path)

url: str = os.environ.get("SUPABASE_URL")
key: str = os.environ.get("SUPABASE_KEY")

if not url or not key:
    raise ValueError("SUPABASE_URL ou SUPABASE_KEY não encontradas no
.env")

# Cliente global do Supabase
supabase: Client = create_client(url, key)
==================================================

Arquivo: backend\app\main.py
Conteúdo:
import shutil
import logging
from pathlib import Path
from typing import List, Tuple, Any
from fastapi import FastAPI, HTTPException, status, UploadFile, File
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware

# Importações internas
```

```
from app.auth.auth_service import auth_service
from app.services.rag_service import rag_service

# Configuração de Logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("DrTilapiaAPI")

# 1. Inicialização do App
app = FastAPI(
    title="Dr. Tilápia 2.0 API",
    version="2.0.1"
)

# 2. Configuração de CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


# 3. Modelos de Dados
class LoginRequest(BaseModel):
    email: str
    password: str


class ChatRequest(BaseModel):
    message: str
    # Usamos List[List[str]] para compatibilidade total com JSON
    history: List[List[str]] = []


# 4. Configurações de Diretório
UPLOAD_DIR = Path("temp_uploads")
UPLOAD_DIR.mkdir(exist_ok=True)


# --- ENDPOINTS ---

@app.get("/health")
async def health_check():
    return {"status": "online", "version": app.version}


@app.post("/auth/login")
async def login(data: LoginRequest):
    result = await auth_service.login(data.email, data.password)
    if not result:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="E-mail ou senha incorretos"
        )
    return result
```

```
@app.post("/admin/upload")
async def upload_document(file: UploadFile = File(...)):
    temp_file = UPLOAD_DIR / file.filename
    try:
        with temp_file.open("wb") as buffer:
            shutil.copyfileobj(file.file, buffer)

        num_chunks = await rag_service.ingest_pdf(str(temp_file),
file.filename)

        return {
            "message": f"Arquivo {file.filename} processado com
sucesso!",
            "chunks_criados": num_chunks
        }
    except Exception as e:
        logger.error(f"Erro no upload: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        if temp_file.exists():
            temp_file.unlink()


@app.post("/consultoria/chat")
async def chat(data: ChatRequest):
    """Endpoint de consulta RAG."""
    try:
        logger.info(f"Nova consulta RAG: {data.message}")

        # Converte o histórico (lista de listas) para lista de tuplas
para o LangChain
        formatted_history = [tuple(h) for h in data.history]

        response = await rag_service.get_answer(data.message,
formatted_history)
        return response
    except Exception as e:
        logger.error(f"Erro no chat: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Erro interno:
{str(e)}")


if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)
==================================================
Diretório: backend\app\auth
--------------------------------------------------

Arquivo: backend\app\auth\__init__.py
Conteúdo:
"""
Módulo de autenticação do DrTilápia 2.0.
"""
==================================================

Arquivo: backend\app\auth\auth_service.py
```

Conteúdo:
```
from datetime import datetime, timedelta
from jose import JWTError, jwt
import os
from app.database import supabase

SECRET_KEY = os.environ.get("JWT_SECRET")
ALGORITHM = os.environ.get("ALGORITHM", "HS256")


class AuthService:
    @staticmethod
    async def login(email: str, password: str):
        try:
            # Autenticação nativa do Supabase
            response = supabase.auth.sign_in_with_password({
                "email": email,
                "password": password
            })

            if response.user:
                return {
                    "access_token": response.session.access_token,
                    "token_type": "bearer",
                    "user": {
                        "id": response.user.id,
                        "email": response.user.email,
                        "role": response.user.user_metadata.get("role",
"user")
                    }
                }
        except Exception as e:
            return None


auth_service = AuthService()
```
==========
Diretório: backend\app\auth__pycache__

Diretório: backend\app\services

Arquivo: backend\app\services__init__.py
Conteúdo:

==========

Arquivo: backend\app\services\rag_service.py
Conteúdo:
```
import os
import logging
from typing import List, Tuple, Dict, Any
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import SupabaseVectorStore
from langchain.chains import ConversationalRetrievalChain
from app.database import supabase
```

```
# Configuração de Logging para monitoramento no Docker
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("RAGService")


class RAGService:
    def __init__(self):
        """
        Inicializa os componentes de IA e a conexão com o banco vetorial.
        """
        # Embeddings: Transforma texto em vetores numéricos
        self.embeddings = OpenAIEmbeddings(model="text-embedding-3-
small")

        # LLM: O cérebro que gera as respostas (GPT-4o)
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)

        # Vector Store: Conexão com a tabela 'documents' no Supabase
        self.vector_store = SupabaseVectorStore(
            client=supabase,
            embedding=self.embeddings,
            table_name="documents",
            query_name="match_documents",
        )

    async def ingest_pdf(self, file_path: str, filename: str) -> int:
        """
        Processa um PDF: Carrega -> Divide em Chunks -> Vetoriza ->
Salva.
        """
        try:
            logger.info(f"Iniciando processamento do PDF: {filename}")

            # 1. Carregamento
            loader = PyPDFLoader(file_path)
            documents = loader.load()

            # 2. Divisão em pedaços (Chunking)
            text_splitter = RecursiveCharacterTextSplitter(
                chunk_size=1000,
                chunk_overlap=100,
                separators=["\n\n", "\n", ".", " ", ""]
            )
            chunks = text_splitter.split_documents(documents)

            # 3. Adição de Metadados para rastreabilidade
            for chunk in chunks:
                chunk.metadata = {
                    "source": filename,
                    "type": "documentacao_tecnica"
                }

            # 4. Persistência no pgvector
            self.vector_store.add_documents(chunks)

            logger.info(f"Sucesso: {len(chunks)} chunks salvos para
{filename}")
```

```
            return len(chunks)

        except Exception as e:
            logger.error(f"Falha na ingestão do arquivo {filename}:
{str(e)}")
            raise e

    async def get_answer(self, question: str, chat_history:
List[Tuple[str, str]] = []) -> Dict[str, Any]:
        """
        Realiza a busca semântica e gera a resposta baseada no contexto.
        """
        try:
            logger.info(f"Processando pergunta: {question}")

            # Configura a Chain de Recuperação
            chain = ConversationalRetrievalChain.from_llm(
                llm=self.llm,

retriever=self.vector_store.as_retriever(search_kwargs={"k": 4}),
                return_source_documents=True
            )

            # Executa a busca e geração
            result = chain.invoke({
                "question": question,
                "chat_history": chat_history
            })

            # Extrai as fontes únicas encontradas
            sources = list(set([
                doc.metadata.get("source", "Fonte desconhecida")
                for doc in result["source_documents"]
            ]))

            return {
                "answer": result["answer"],
                "sources": sources
            }

        except Exception as e:
            # ALTERAÇÃO PARA DEBUG: Retornando o erro real no corpo da
resposta
            logger.error(f"CRITICAL ERROR no RAG: {str(e)}")
            return {
                "answer": f"Erro Técnico Detectado: {str(e)}",
                "sources": [],
                "debug_info": "Verifique a API Key da OpenAI ou a função
match_documents no Supabase."
            }


# Instância Singleton
rag_service = RAGService()
==================================================
Diretório: backend\app\services\__pycache__
--------------------------------------------------
Diretório: backend\app\__pycache__
```

```
--------------------------------------------------
Diretório: backend\docs
--------------------------------------------------
Diretório: backend\temp_uploads
--------------------------------------------------
Diretório: backend\__pycache__
--------------------------------------------------
Diretório: frontend
--------------------------------------------------

Arquivo: frontend\rxconfig.py
Conteúdo:
import reflex as rx

config = rx.Config(
    # Nome do pacote onde reside a lógica da UI (pasta 'ui')
    app_name="ui",

    # URL pública da sua API FastAPI (Motor de IA)
    # O Frontend usará este endereço para chamadas httpx
    api_url="http://localhost:8000",

    # Porta interna que o Reflex usa para gerenciar o estado (Eventos)
    # Definimos 8001 para evitar conflito com o FastAPI na 8000
    backend_port=8001,

    # Banco de dados interno do Reflex (para persistência de estado da
UI)
    db_url="sqlite:///reflex.db",

    # Configuração de CORS para permitir conexões do navegador
    cors_allowed_origins=["*"]
)
==================================================
Diretório: frontend\assets
--------------------------------------------------
Diretório: frontend\ui
--------------------------------------------------

Arquivo: frontend\ui\__init__.py
Conteúdo:

==================================================

Arquivo: frontend\ui\state.py
Conteúdo:
import reflex as rx
import httpx
import json
from typing import List, Tuple


class State(rx.State):
    # Lista de mensagens: [("pergunta", "resposta"), ...]
    chat_history: List[List[str]] = []
    current_message: str = ""
    is_loading: bool = False
```

```
    async def answer(self):
        if not self.current_message:
            return

        self.is_loading = True
        yield  # Atualiza a UI para mostrar o estado de carregamento

        # Prepara a pergunta e limpa o campo de input
        user_question = self.current_message
        self.current_message = ""

        # Adiciona a pergunta do usuário ao chat imediatamente
        self.chat_history.append([user_question, "Pensando..."])
        yield

        try:
            async with httpx.AsyncClient(timeout=60.0) as client:
                response = await client.post(
                    "http://localhost:8000/consultoria/chat",
                    json={
                        "message": user_question,
                        "history": self.chat_history[:-1]  # Envia o
histórico anterior
                    }
                )

                if response.status_code == 200:
                    data = response.json()
                    # Atualiza a última mensagem com a resposta real da
IA
                    self.chat_history[-1][1] = data["answer"]
                else:
                    self.chat_history[-1][1] = "Erro ao conectar com o
consultor."
        except Exception as e:
            self.chat_history[-1][1] = f"Erro de conexão: {str(e)}"

        self.is_loading = False
        yield
==================================================

Arquivo: frontend\ui\styles.py
Conteúdo:
# frontend/ui/styles.py
# Paleta de cores
PRIMARY = "#004e92"
ACCENT = "#ff6b6b"
SECONDARY = "#4ecdc4"
WHITE = "#ffffff"
TEXT_MUTED = "#b0b0b0"

# Gradiente de fundo
BACKGROUND_GRADIENT = "linear-gradient(135deg, #004e92 0%, #000000 100%)"

# Estilo do fundo do herói
HERO_BG_STYLE = {
    "background_image": "url('/hub01.jpeg')",
    "background_size": "cover",
```

```
    "background_position": "center",
    "background_repeat": "no-repeat",
    "background_attachment": "fixed",
}

# Estilo do container de vidro
GLASS_CONTAINER_STYLE = {
    "background_color": "rgba(255, 255, 255, 0.12)",
    "backdrop_filter": "blur(12px)",
    "border_radius": "12px",
    "padding": "20px",
    "box_shadow": "0 4px 6px rgba(0, 0, 0, 0.1)",
}

# Estilo de input
INPUT_STYLE = {
    "background_color": "rgba(0, 0, 0, 0.7)",
    "color": WHITE,
    "border": f"1px solid {ACCENT}",
    "border_radius": "8px",
    "padding": "10px",
    "font_size": "16px",
    "_focus": {
        "border_color": PRIMARY,
        "box_shadow": f"0 0 0 1px {PRIMARY}",
    },
}

# Estilo do botão de destaque
BUTTON_ACCENT = {
    "background_color": ACCENT,
    "color": WHITE,
    "border_radius": "8px",
    "padding": "12px 24px",
    "font_size": "16px",
    "font_weight": "bold",
    "cursor": "pointer",
    "transition": "all 0.3s ease",
    "_hover": {
        "background_color": "#ff5252",
        "transform": "translateY(-2px)",
        "box_shadow": "0 4px 8px rgba(255, 107, 107, 0.3)",
    },
}
==================================================

Arquivo: frontend\ui\ui.py
Conteúdo:
import reflex as rx
from ui.state import State
from ui.pages.index import index
from ui.pages.login import login_page
from ui.pages.hub import hub
from ui.pages.consultoria import consultoria
from ui.pages.admin_rag import admin_rag
from ui.pages.dashboard import dashboard

app = rx.App(
```

```
    theme=rx.theme(appearance="dark", accent_color="blue")
)

# Registro de Rotas
app.add_page(index, route="/")
app.add_page(login_page, route="/login")
app.add_page(hub, route="/hub", on_load=State.check_login)
app.add_page(consultoria, route="/consultoria",
on_load=State.check_login)
app.add_page(admin_rag, route="/admin-rag", on_load=State.check_login)
app.add_page(dashboard, route="/dashboard", on_load=State.check_login)
==================================================
Diretório: frontend\ui\pages
--------------------------------------------------

Arquivo: frontend\ui\pages\admin_rag.py
Conteúdo:
from ui.state import State
from ui.styles import GLASS_CONTAINER_STYLE, ACCENT, WHITE,
BACKGROUND_GRADIENT

import reflex as rx


def admin_rag() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.heading(
                "Administração do RAG - DrTilapIA 2.0",
                color=WHITE,
                size="lg",
            ),
            rx.upload(
                rx.button(
                    "Selecionar PDFs",
                    color=WHITE,
                    bg=ACCENT,
                ),
                id="upload_pdfs",
                multiple=True,
                accept={".pdf": "application/pdf"},
            ),
            rx.button(
                "Fazer Upload",

on_click=State.handle_upload(rx.upload_files(id="upload_pdfs")),
                color=WHITE,
                bg=ACCENT,
            ),
            rx.vstack(
                rx.text("Arquivos selecionados:", color=WHITE),
                rx.foreach(State.files_to_upload, lambda file:
rx.text(file, color=WHITE)),
            ),
            rx.button(
                "Reindexar Base de Conhecimento",
                on_click=State.reindex_vectorstore,
                color=WHITE,
```

```
                bg=ACCENT,
            ),
            rx.cond(
                State.upload_status,
                rx.text(State.upload_status, color=WHITE),
            ),
            rx.cond(
                State.reindex_status,
                rx.text(State.reindex_status, color=WHITE),
            ),
            spacing="4",
            align="center",
        ),
        style={
            **GLASS_CONTAINER_STYLE,
            "bg": "rgba(0, 0, 0, 0.4)",
            "backdrop_filter": "blur(10px)",
        },
        height="100vh",
        width="100%",
        bg=BACKGROUND_GRADIENT,
    )
```

==========

Arquivo: frontend\ui\pages\consultoria.py
Conteúdo:

```
import reflex as rx
from ..state import State
from ..styles import PRIMARY, ACCENT, SECONDARY


def message_box(chat: list) -> rx.Component:
    """Renderiza uma bolha de mensagem individual."""
    return rx.vstack(
        # Pergunta do Usuário
        rx.box(
            rx.text(chat[0], color="white", font_weight="500"),
            background="rgba(255, 255, 255, 0.1)",
            backdrop_filter="blur(10px)",
            border=f"1px solid rgba(255, 255, 255, 0.2)",
            padding="1rem",
            border_radius="15px 15px 0px 15px",
            align_self="flex-end",
            max_width="80%",
        ),
        # Resposta da IA
        rx.box(
            rx.vstack(
                rx.text(chat[1], color="white"),
                rx.cond(
                    chat[1] != "Pensando...",
                    rx.hstack(
                        rx.icon(tag="info", size=14, color=SECONDARY),
                        rx.text("Baseado em manuais técnicos",
font_size="0.7rem", color=SECONDARY),
                        margin_top="0.5rem",
                    ),
                ),
```

```
                align_items="start",
            ),
            background=f"linear-gradient(135deg, {PRIMARY}88,
#00000088)",
            backdrop_filter="blur(12px)",
            border=f"1px solid {SECONDARY}44",
            padding="1rem",
            border_radius="15px 15px 15px 0px",
            align_self="flex-start",
            max_width="85%",
            margin_top="0.5rem",
        ),
        width="100%",
        spacing="1",
    )


def consultoria_page() -> rx.Component:
    return rx.center(
        # Background Imersivo
        rx.box(
            position="fixed",
            top="0",
            left="0",
            width="100%",
            height="100%",
            background="url('/bg_hero.png')",
            background_size="cover",
            background_position="center",
            filter="brightness(0.4) blur(8px)",
            z_index="-1",
        ),

        rx.vstack(
            # Header da Página
            rx.hstack(
                rx.image(src="/favicon.ico", width="40px",
height="auto"),
                rx.heading("Consultor Dr. Tilápia", size="7",
color="white"),
                rx.spacer(),
                rx.badge("RAG Ativo", color_scheme="cyan",
variant="outline"),
                width="100%",
                padding_bottom="2rem",
            ),

            # Área de Chat (Scrollable)
            rx.scroll_area(
                rx.vstack(
                    rx.foreach(State.chat_history, message_box),
                    width="100%",
                    spacing="4",
                    padding="1rem",
                ),
                height="60vh",
                width="100%",
                background="rgba(0, 0, 0, 0.3)",
```

```
                border_radius="20px",
                border="1px solid rgba(255, 255, 255, 0.1)",
                margin_bottom="1.5rem",
            ),

            # Área de Input
            rx.hstack(
                rx.input(
                    placeholder="Digite sua dúvida técnica (ex: Qual a
temperatura ideal?)...",
                    value=State.current_message,
                    on_change=State.set_current_message,
                    on_key_down=lambda e: rx.cond(e.key == "Enter",
State.answer),
                    background="rgba(255, 255, 255, 0.05)",
                    border=f"1px solid rgba(255, 255, 255, 0.2)",
                    color="white",
                    _focus={"border": f"1px solid {SECONDARY}"},
                    height="3.5rem",
                    flex_grow="1",
                    border_radius="12px",
                ),
                rx.button(
                    rx.cond(
                        State.is_loading,
                        rx.spinner(size="2"),
                        rx.icon(tag="send"),
                    ),
                    on_click=State.answer,
                    background=f"linear-gradient(45deg, {PRIMARY},
{SECONDARY})",
                    color="white",
                    _hover={"transform": "scale(1.05)", "opacity":
"0.9"},
                    height="3.5rem",
                    width="4rem",
                    border_radius="12px",
                    is_disabled=State.is_loading,
                ),
                width="100%",
                spacing="3",
            ),

            # Rodapé informativo
            rx.text(
                "O Dr. Tilápia utiliza IA para analisar manuais. Sempre
valide com um técnico local.",
                font_size="0.7rem",
                color="rgba(255, 255, 255, 0.5)",
                margin_top="1rem",
            ),

            width="100%",
            max_width="900px",
            padding="2rem",
            background="rgba(255, 255, 255, 0.02)",
            backdrop_filter="blur(20px)",
            border_radius="30px",
```

```
            border="1px solid rgba(255, 255, 255, 0.1)",
            box_shadow="0 8px 32px 0 rgba(0, 0, 0, 0.8)",
        ),
        width="100%",
        min_height="100vh",
        padding_y="2rem",
    )
```

==========

Arquivo: frontend\ui\pages\dashboard.py
Conteúdo:

```
import reflex as rx
from ui.styles import GLASS_CONTAINER_STYLE, ACCENT, WHITE,
BACKGROUND_GRADIENT, SECONDARY

def metric(label, val, unit):
    return rx.vstack(
        rx.text(label, font_size="0.8em", color="rgba(255,255,255,0.6)"),
        rx.hstack(rx.text(val, font_size="2em", font_weight="bold",
color=WHITE), rx.text(unit, color=SECONDARY)),
        bg="rgba(0,0,0,0.2)", padding="1.5em", border_radius="1em",
width="180px"
    )

def dashboard() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.heading("Métricas em Tempo Real", color=WHITE),
            rx.flex(
                metric("Temperatura", "26.5", "°C"),
                metric("Oxigênio", "7.2", "mg/L"),
                metric("pH", "6.8", "pH"),
                gap="1.5em", flex_wrap="wrap", justify="center"
            ),
            rx.link("Voltar ao Hub", href="/hub", color=WHITE,
margin_top="2em"),
            width="800px", padding="3em", **GLASS_CONTAINER_STYLE
        ),
        height="100vh", width="100%", background=BACKGROUND_GRADIENT
    )
```

==========

Arquivo: frontend\ui\pages\hub.py
Conteúdo:

```
import reflex as rx
from ui.state import State
from ui.styles import WHITE, ACCENT, GLASS_CONTAINER_STYLE


def floating_card(title: str, desc: str, href: str, icon: str, **kwargs):
    return rx.link(
        rx.vstack(
            rx.icon(tag=icon, size=30, color=ACCENT),
            rx.text(title, font_weight="bold", color=WHITE),
            rx.text(desc, font_size="0.8em",
color="rgba(255,255,255,0.6)"),
            spacing="2", align="center",
        ),
```

```
        href=href, padding="1.5em", **GLASS_CONTAINER_STYLE,
position="absolute", **kwargs
    )


def hub() -> rx.Component:
    return rx.box(
        rx.image(src="/hub01.jpeg", width="100%", height="100vh",
object_fit="cover", position="absolute", z_index="-1",
                 filter="brightness(0.4)"),
        rx.center(rx.heading(f"Olá, {State.user_name}", size="8",
color=WHITE, position="absolute", top="10%")),

        floating_card("Consultoria", "Chat IA", "/consultoria", "message-
square", top="40%", left="15%"),
        floating_card("Dashboard", "Métricas", "/dashboard", "bar-chart-
3", top="40%", right="15%"),

        rx.cond(
            State.user_role == "admin",
            floating_card("Admin RAG", "Gestão", "/admin-rag",
"database", bottom="15%", left="43%"),
        ),

        rx.button("Sair", on_click=State.logout, position="absolute",
top="2em", right="2em", variant="ghost",
                  color=WHITE),
        height="100vh", width="100%", position="relative"
    )
==================================================

Arquivo: frontend\ui\pages\index.py
Conteúdo:
import reflex as rx
from ui.styles import HERO_BG_STYLE, WHITE, BUTTON_ACCENT

def index() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.heading("Dr. Tilápia 2.0", color=WHITE, size="9"),
            rx.button("Entrar no Sistema",
on_click=rx.redirect("/login"), **BUTTON_ACCENT),
            align="center",
            spacing="6",
            padding="2em",
        ),
        style=HERO_BG_STYLE,
        height="100vh",
        width="100%",
    )
==================================================

Arquivo: frontend\ui\pages\login.py
Conteúdo:
import reflex as rx
from ui.state import State
from ui.styles import WHITE, GLASS_CONTAINER_STYLE, BUTTON_ACCENT,
PRIMARY
```

```
def login_page():
    return rx.center(
        rx.vstack(
            rx.heading("Dr. Tilápia 2.0", size="8", color=WHITE),
            rx.input(placeholder="E-mail",
on_change=State.set_user_email, width="100%",
bg="rgba(255,255,255,0.05)"),
            rx.input(placeholder="Senha", type="password",
on_change=State.set_password, width="100%", bg="rgba(255,255,255,0.05)"),
            rx.button("Entrar", on_click=State.handle_login,
is_loading=State.is_loading, width="100%", **BUTTON_ACCENT),
            rx.text(State.error_message, color="red"),
            spacing="4", padding="3em", width="400px",
**GLASS_CONTAINER_STYLE
        ),
        height="100vh", width="100%", background=f"radial-
gradient(circle, {PRIMARY}, #000)"
    )
==================================================
Diretório: frontend\ui\pages\__pycache__
--------------------------------------------------
Diretório: frontend\ui\__pycache__
--------------------------------------------------
Diretório: frontend\uploaded_files
--------------------------------------------------
Diretório: frontend\__pycache__
--------------------------------------------------
```